Sơ đồ luồng xử lý chi tiết — Nginx, Redis & chia tải 2 tài khoản Cloudflare

Backend HoamyTech · production `api.hoamytech.com` · dev `api-dev.hoamytech.com` · cập nhật 11/11/2026 · trạng thái pool theo `.env.local` (chưa đối chiếu cấu hình thật trên VPS)

1. Kiến trúc tổng thể (production)

App Flutter / Web client
Thu âm, gửi audio · nhận bản dịch + `audioUrl` · hội thoại, luyện nghe, dịch text, TTS

↓ HTTPS 443

① Nginx host (trên VPS, ngoài Docker)
Giữ chứng chỉ HTTPS · `api.hoamytech.com` → Docker prod · `api-dev.hoamytech.com` → Docker dev (cổng riêng)

↓ HTTP nội bộ

② Nginx Docker — Load balancer MỌI API

- **least_conn**: backend nào đang ít kết nối hơn thì nhận request.
- Backend lỗi 2 lần trong 10 giây → tạm bỏ qua 10 giây (`max_fails=2 fail_timeout=10s`).
- Lỗi kết nối / timeout → tự thử backend còn lại (`proxy_next_upstream_tries 2`).
- Ngoại lệ: `/api/audio/stream?node=3000|3001` → ghim về đúng backend đã tạo TTS (backend kia làm dự phòng).
- Giới hạn body 32 MB · chờ phản hồi tối đa 60 giây.

↙ ↘

backend-3000
Next.js · 1 GB RAM · 1 CPU
Dispatcher Cloudflare riêng trong tiến trình (tối đa 20 request audio đang xử lý, 40 chờ)

backend-3001
Next.js · 1 GB RAM · 1 CPU
Giống hệt backend-3000; 2 bên phối hợp hạn mức qua Redis

audio-worker
Xử lý job dịch bất đồng bộ
JOBS ĐANG TẮT

↓ Cả 2 backend dùng chung

REDIS PRODUCTION — trạng thái tạm dùng chung (dev có bộ Redis riêng)

redis — Quota pool Cloudflare
- Đếm lượt/phút (RPM), burst, số request đồng thời cho từng tài khoản và từng model
- Cấp "permit" nguyên tử: 2 backend không thể cùng vượt hạn mức
- Cooldown khi Cloudflare trả 429 / lỗi tạm thời

Namespace `speaking-ai-cf-quota-v1` · 256 MB

redis-cache — Cache kết quả
Kết quả dịch audio (khoảng 5 phút)
Lặp lại: **0,29 giây** thay vì 7,2 giây
128 MB · mất cũng không sao

redis-jobs — Hàng đợi job
Dịch bất đồng bộ (có volume lưu bền)
ĐANG TẮT
256 MB

↓ Gọi ra ngoài

DỊCH VỤ BÊN NGOÀI

Cloudflare Workers AI — 2 tài khoản

audio-a TÀI KHOẢN CŨ
= `CLOUDFLARE_ACCOUNT_ID`
ASR (Whisper) · Dịch (Llama, Qwen) · **TTS** (Aura, Grok)
Phục vụ MỌI tính năng

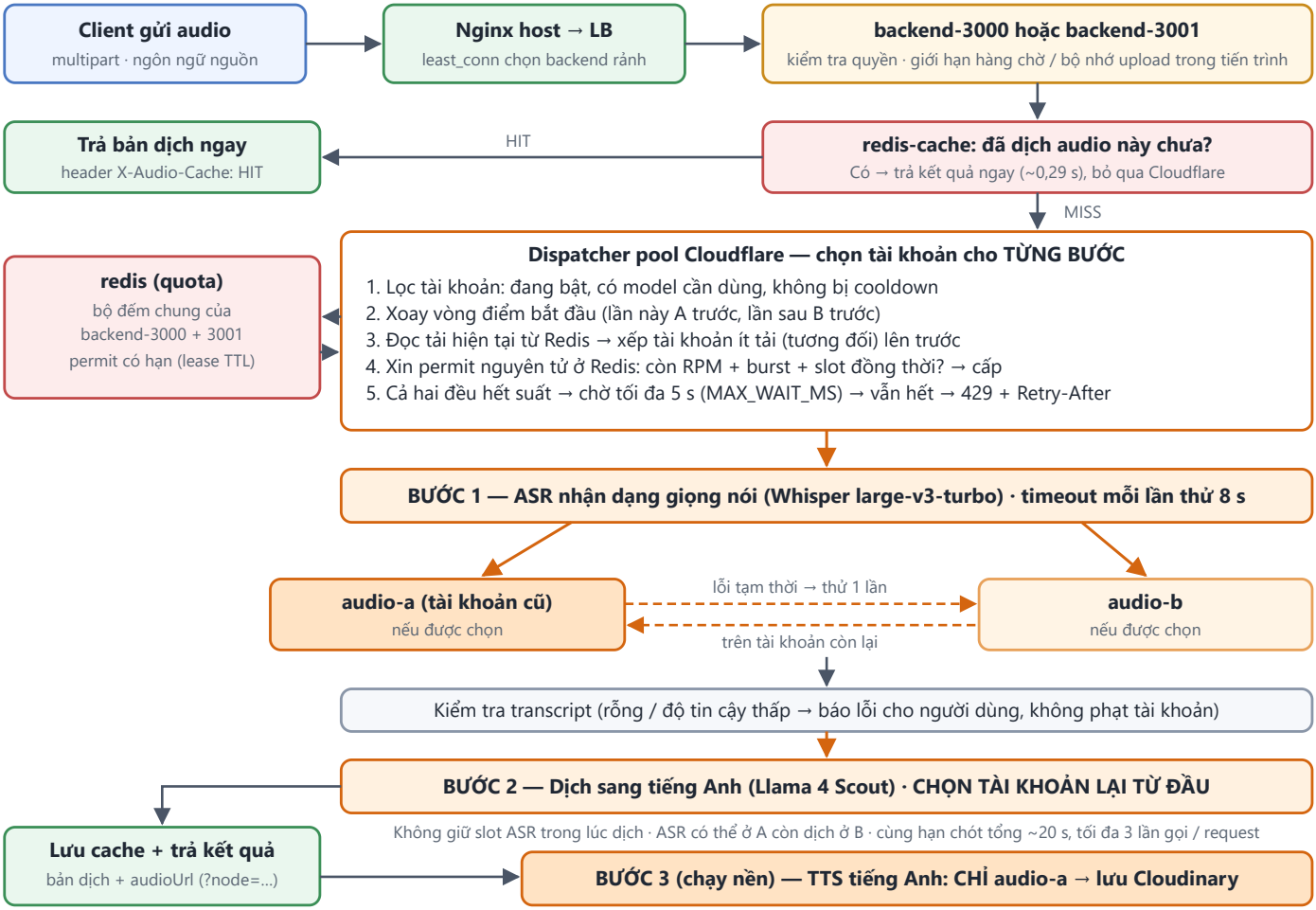
audio-b PHỤ
Chỉ ASR (Whisper) · Dịch (Llama)
Không có TTS · chỉ nhận `/api/audio/translate`

Neon PostgreSQL
Dữ liệu chính: thiết bị, lịch sử, phiên audio
DEV DÙNG CHUNG

Cloudinary
Lưu file audio TTS để dùng lại

Hai loại "cân bằng tải" khác nhau: (1) **Nginx** chia request giữa `backend-3000` và `backend-3001` cho **mọi API**. (2) **Pool Cloudflare** chia lượt gọi AI giữa **audio-a** và **audio-b**, nhưng **chỉ cho ASR và bước dịch của** `/api/audio/translate`. Mọi API khác và mọi TTS chỉ chạy trên audio-a. Nét đứt = thành phần đang tắt.

2. Luồng chi tiết `POST /api/audio/translate` — có chia tải 2 tài khoản



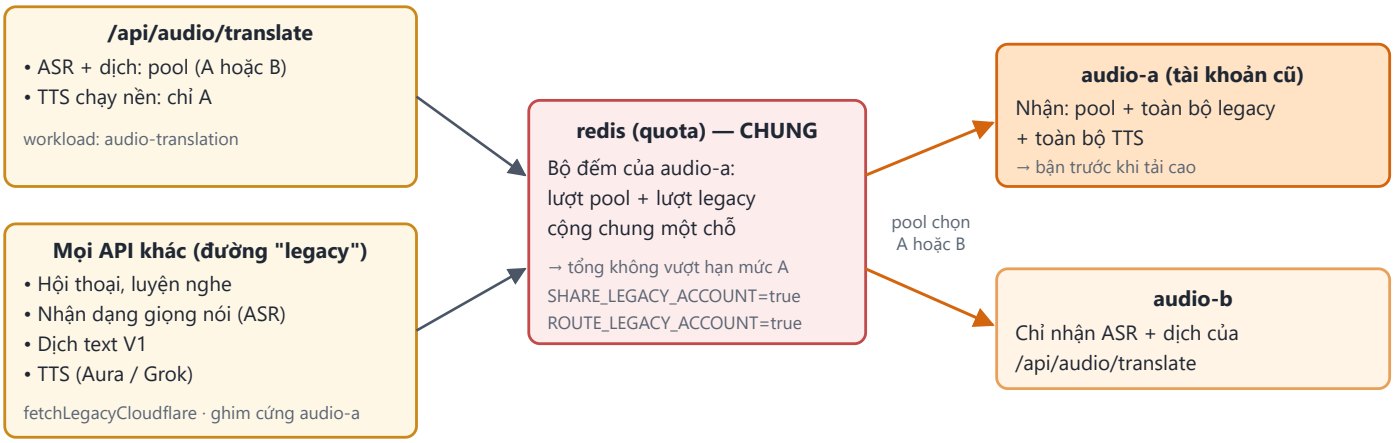
Diễn giải từng bước

1. Client gửi audio qua HTTPS → Nginx host → load balancer chọn backend đang ít kết nối hơn.
2. Backend kiểm tra quyền, giới hạn hàng chờ và tổng dung lượng upload đang giữ (32 MB) để không bị tràn RAM khi đông người.
3. Xem **redis-cache**: audio đã dịch gần đây thì trả ngay, không tốn lượt Cloudflare.
4. Chưa có: dispatcher **chọn tài khoản cho bước ASR** (xoay vòng, ưu tiên tài khoản ít tải), xin permit ở **redis** rồi gọi Whisper.
5. Kiểm tra transcript, rồi **chọn tài khoản lại cho bước dịch**, độc lập với bước ASR.
6. Lưu kết quả vào cache, trả bản dịch kèm `audioUrl` có `?node=3000|3001`.
7. Song song: tạo giọng đọc tiếng Anh **chỉ trên audio-a**; client mở `audioUrl` → nginx đưa về đúng backend đang tạo → nghe ngay; file được lưu lên Cloudinary.

Khi một tài khoản gặp sự cố

Tình huống	Dispatcher xử lý
Timeout, lỗi mạng, HTTP 5xx	Thử lại **1 lần trên tài khoản chưa dùng**; tạm tránh model đó trên tài khoản lỗi khoảng 1 giây
429 (Cloudflare báo vượt hạn mức)	Cooldown tài khoản theo `Retry-After`, chuyển sang tài khoản kia nếu còn lượt
Token sai / hết quyền	Tắt tài khoản đó khỏi pool, dùng tài khoản còn lại
Cả 2 hết suất	Chờ tối đa 5 s; vẫn hết → trả **429** kèm `Retry-After`
Hết hạn chót tổng (~20 s)	Trả **504**; không thử thêm
Redis quota lỗi	Trả lỗi có kiểm soát; **không** gọi thẳng Cloudflare bỏ qua hạn mức
Client ngắt kết nối	Huỷ lượt gọi đang chạy, trả permit

3. Các API còn lại — chỉ dùng tài khoản cũ (audio-a), nhưng chung bộ đếm hạn mức



API / tính năng	Nginx chia giữa 2 backend	Chia giữa audio-a / audio-b	Tính chung hạn mức qua Redis
`/api/audio/translate` — ASR, dịch	Có	**Có**	Có
`/api/audio/translate` — TTS chạy nền	Có (stream ghim theo `?node=`)	Không — chỉ audio-a	Có
Hội thoại, luyện nghe, ASR, dịch text, TTS	Có	Không — chỉ audio-a	Có

4. Production và dev trên cùng VPS

Production
Nginx Docker + `backend-3000`, `backend-3001`, `audio-worker` · 3 Redis riêng · deploy từ nhánh `can-bang-tai`

Dev
Nginx Docker + 1 `backend` (1 GB, 0,75 CPU) · 3 Redis riêng · deploy từ nhánh `dev`

Dùng chung
CPU / RAM / ổ đĩa VPS · phần mềm Docker · Nginx host · **Neon PostgreSQL**

Rủi ro cần nhớ:
• **Hạn mức Cloudflare:** Redis dev và prod tách riêng nên bộ đếm cũng tách. Nếu dev bật pool với cùng 2 tài khoản, mỗi bên tưởng mình có toàn bộ hạn mức → tổng có thể vượt hạn mức thật. Cho dev dùng tài khoản/token khác hoặc đặt `limits` thấp hơn.
• **audio-a là điểm nghẽn:** mọi tính năng cũ và mọi TTS đều dồn vào A. Muốn các API khác cũng được chia tải phải đưa chúng qua dispatcher pool và khai báo TTS cho audio-b.
• **Database dùng chung:** dev ghi/xoá hoặc chạy migration sẽ tác động thẳng vào dữ liệu thật của prod.

Thông số đang dùng (theo `.env.example` / `.env.local`): tối đa 2 lần thử mỗi bước, 3 lần mỗi request · chờ suất tối đa 5 s · 20 request audio đang xử lý và 40 request chờ mỗi backend · bộ nhớ upload 32 MB mỗi backend · timeout ASR mỗi lần thử 8 s, dịch 6 s, hạn chót tổng ~20 s · thử lại lỗi tạm thời bật (`CLOUDFLARE_POOL_RETRY_TRANSIENT=true`, có thể bị tính tiền 2 lần ở Cloudflare).